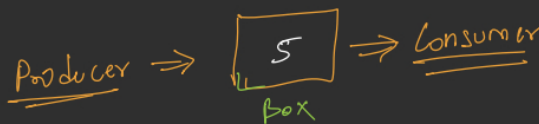


Inter Thread Communication

Object

- ↳ wait()
- ↳ notify()
- ↳ notifyAll()

Producer-Consumer Problem



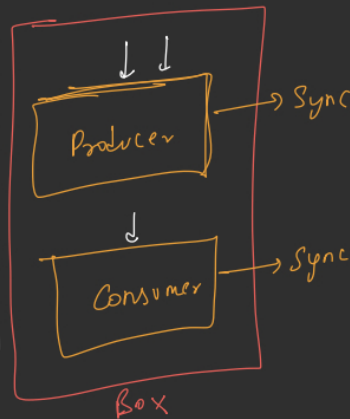
(Non-deterministic)

- ① Consumer tries to consume Null value.
- ② Producer produces a value w/o Consumer consumed the prev. one.

Synchronized $\{$

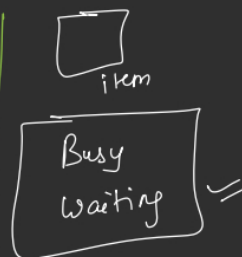
① — produce

② $\rightarrow$ consume



```
producer(int value) {  
    while (item != null) {  
         $\rightarrow$  wait();  
    }  
    item = value;  
    System.out.println(value);  
}
```

```
consumer() {  
    while (item == null) {  
         $\rightarrow$  wait();  
    }  
    s = c.item;  
    item = null;  
}
```



↳ WAITING

↳ wait()
→ notify()
→ notifyAll()

Thread Communication (Problem)

↳ Shared Resource. (Box) → item
↳ Condition. (box has item or not)
↳ Waiting (Consumer wait → item (x)
Producer wait → item (✓))

wait()

- ① Release Monitor locks.
- ② It goes in WAITING state.
- ③ It stays there, until another thread wakes it up.

Consumer

↳ while (flag == false) {
wait();
item = null;
notify();

notify() ✓

notifyAll() ✓

Producer

↳ while (flag == true) {
wait();
item = value;
flag = true;
item = (item);
notify();

wait() → Synchronized()

↳ Exception → IllegalMonitorStateException

```
Synchronized void produce (val) {  
    while (flag == true) {  
        wait();  
    }  
}
```

```
① → Synchronized void produce (val) {  
    while (val == true) {  
        wait();  
    }  
    item = val;  
    flag = true;  
    notify();  
}  
  
② → ② → Synchronized void consume () {  
    while (val == false) {  
        wait();  
    }  
    item = null;  
    flag = false;  
    notify();  
}
```

flag
false

item

Box

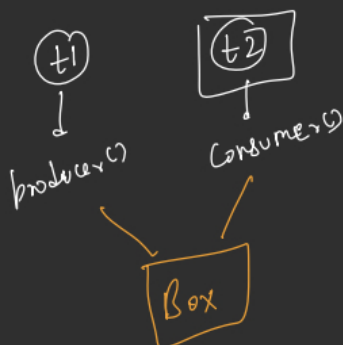
Queue

Object

↳ Monitor locks

BLOCKED

↳ OwnerThread : null
↳ Waiting Queue = [,]



notify()

notify()

→ synchronized() →

→ wake up a random thread
from waiting Queue.

Non
deterministic

- ① One Random Thread is picked from waiting Queue.
- ② That Thread → BLOCKED state.
- ③ Compete for the lock.
- ④ Once lock occupied → RUNNING.

Exception : →

✓✓

notifyAll()

- ① All threads in waiting Queue are moved to BLOCKED state.
- ② They all try to acquire the lock.
- ③ Only one gets the lock at a time.

P2

```

synchronize void producer (val) {
    if (flag == true) {
        wait();
    }
    item = val;
    flag = true;
    notify();
    notifyAll();
}

```

C2

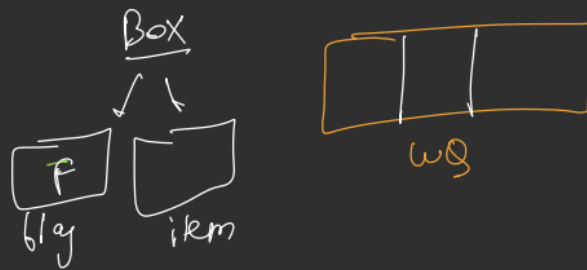
```

synchronize void consumer() {
    if (flag == false) {
        wait();
    }
    item = null;
    flag = false;
    notify();
    notifyAll();
}

```

notifyAll()

if (flag == false)



notify()

Deadlock
problem

notifyAll()



(Expensive)

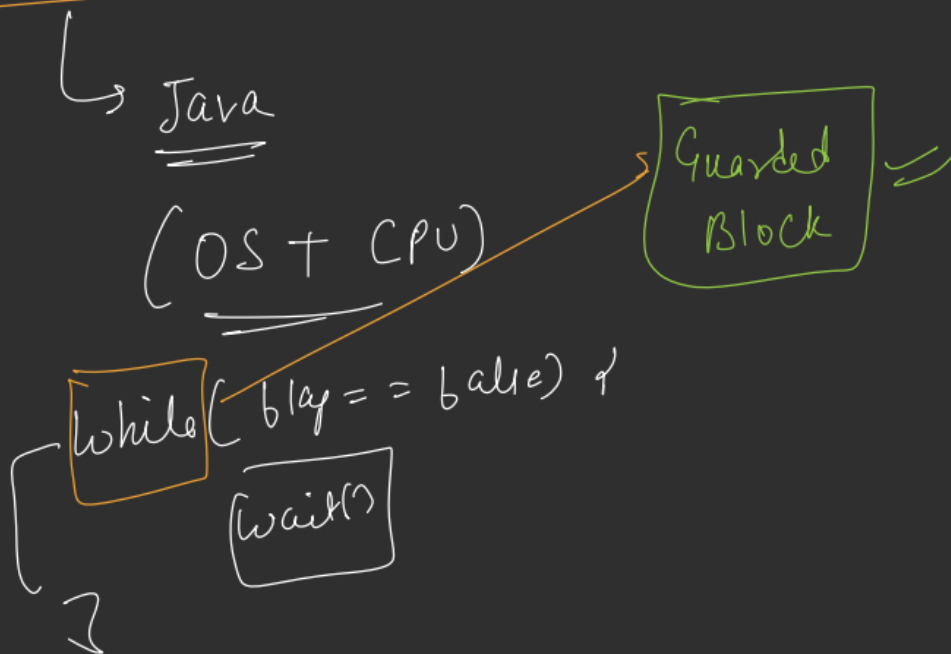
Overhead.

wait(), notify(), notifyAll()

Object

<u>Sleep()</u> ↓ <u>TIMED-WAITING</u> sleep(200)	<u>wait()</u> ↓ <u>WAITING</u>
---	--

Spurious Wakeup



Producer → Consumer
 Doc Creator → Printer

